



Coursework: Sistema de Ticketing – Sunrise Ski Session (Davos Parsenn)

Software Development

Jorge Garrido
Francisco Javier Puig

Ski_ticketing_final_V3.c y ticketing.py

José Palacios Boertegui

Bachelor in Applied Computing

10/12/2025

Tabla de contenido

1. Introducción y objetivos.....	3
2. Contexto del negocio: Davos Klosters / Sunrise Ski Session.....	3
3. Requerimientos y reglas de negocio (funcionales + extras).....	3
3.1 Requerimientos mínimos del enunciado (obligatorios).....	3
3.2 Reglas de negocio añadidas (valor agregado).....	4
3.3 Validaciones y control de errores (calidad).....	4
3.4 Casos límite contemplados.....	4
4. Diseño de la solución (arquitectura + decisiones).....	5
4.1 Flujo general por menús (casos de uso).....	5
4.2 Diseño de datos y estructuras.....	5
4.3 Algoritmos y lógica clave.....	5
4.4 Validación y control de errores.....	5
4.5 Valor agregado (diseño de extras).....	5
5. Implementación en C.....	6
5.1 Estructura del programa y modularización.....	6
5.2 Estructuras de datos, arrays y control de aforo.....	6
5.3 Control de flujo y validación de entradas.....	6
5.4 Informes, KPIs y ordenación.....	6
6. Implementación en Python.....	6
6.1 Estructura y arquitectura.....	6
6.2 Persistencia con SQLite y modelo de datos.....	6
6.3 Reglas de negocio, validaciones y seguridad de acceso.....	7
6.4 Funcionalidades extra (valor agregado).....	7
7. Comparación C vs Python.....	7
8. Pruebas y verificación.....	7
9. Guía de usuario.....	8
9.1 Ejecución y uso en C.....	8
9.2 Ejecución y uso en Python.....	8
10. Apéndice.....	9
Apéndice A — Checklist de Requisitos (Trazabilidad).....	9
Apéndice B - Capturas de ejecución / Evidencia de funcionamiento.....	12
Python - Ski_ticketing_final_V3.....	12
C (ski_ticketing_final_v3.c).....	17
APÉNDICE C — Diagrama de flujo / Pseudocódigo.....	23

C.1 Flujo general (C — Sunrise Ski Session)	23
C.2 Flujo general (Python — Ticketing System CLI)	23
Apéndice D — Esquema de Base de Datos (SQLite)	24
D.1 Objetivo del esquema	24
D.2 Configuración SQLite (PRAGMAs)	24
D.3 Esquema principal (Aplicación en C)	25
D.3.2 Tabla event (configuración del evento).....	25

1. Introducción y objetivos

Este trabajo presenta el diseño e implementación de un sistema de *ticketing* para un evento de nieve inspirado en Davos Klosters (Sunrise Ski Session – Davos Parsenn), desarrollado en dos versiones: una en C y otra en Python. El objetivo principal es demostrar competencia en *Software Development* aplicando análisis de requisitos, diseño de menús, estructuras de datos, validación de entradas, control de flujo, gestión de errores y generación de informes.

La implementación en C está orientada a la operativa de venta y reserva con aforo limitado, selección de plaza, aplicación de descuentos y cierre de ventas con KPIs e informes ordenados. La implementación en Python amplía el sistema con persistencia en SQLite y una estructura más escalable (capas/servicios), incorporando funciones extra como auditoría, métricas, exportación y pruebas unitarias.

2. Contexto del negocio: Davos Klosters / Sunrise Ski Session

En una estación como Davos Klosters, la venta de tickets para un evento (p. ej., *Sunrise Ski Session* — *Davos Parsenn*) requiere un sistema operativo simple, rápido y controlado por el negocio. La aplicación se utiliza para iniciar y configurar la venta, lanzar el proceso de compra para clientes y, al final, elaborar informes para el control económico y de aforo.

El flujo parte de un menú de configuración reservado al administrador, que debe introducir un código secreto fijo (CEAC4013) y definir como mínimo: nombre del evento, licencia, aforo y precio unitario. Durante el uso, el sistema debe mostrar siempre el aforo total y las plazas disponibles.

En ventas, el cliente consulta disponibilidad, reserva indicando nombre, edad, descuento (DESC5/10/15) y posición; cada venta registra el total pagado y se cierran ventas al alcanzar el aforo, habilitando el menú de informes.

3. Requerimientos y reglas de negocio (funcionales + extras)

3.1 Requerimientos mínimos del enunciado (obligatorios)

El sistema debe cubrir el flujo completo configuración → venta → cierre → informes. En la configuración, el usuario administrador debe introducir un código secreto fijo e inmodificable: “CEAC4013”, y después definir como mínimo nombre del evento, licencia, aforo y precio unitario. Además, durante la ejecución debe mostrarse de forma visible el aforo total y las plazas disponibles.

Tras configurar, se habilita el menú de ventas con, al menos:

- Listar disponibilidad: mostrar todas las posiciones libres.
- Reservar: volver a listar disponibilidad, pedir nombre, edad, y si aplica descuento. Si hay descuento, solo se admiten DESC5 (–5%), DESC10 (–10%), DESC15 (–15%), en caso contrario no aplica. El cliente elige la posición a reservar y cada venta queda registrada

guardando el total pagado. El sistema debe cerrar ventas automáticamente al alcanzar el aforo y debe existir una opción explícita de “Cerrar y generar informe”.

En el cierre / informes, al elegir “Cerrar y generar informe” (o por aforo completo) se vuelve a solicitar el código secreto y se muestra un menú con:

- Ver informe (suma de tickets + total de dinero),
- Ver ventas ascendente (barato → caro),
- Ver ventas descendente (caro → barato).

3.2 Reglas de negocio añadidas (valor agregado)

Para maximizar nota, se añadieron reglas coherentes con el contexto “Davos Ski Session”, manteniendo el core del enunciado:

- Confirmación de compra antes de registrar la venta (si no confirma, no se guarda).
- Límites operativos: edad mínima (p. ej. MIN_AGE = 14) y máximo de tickets por reserva (MAX_PER_RESERVATION = 4) para simular control de acceso y reducir errores de gestión.
- Persistencia y trazabilidad con SQLite + auditoría + métricas, incluyendo reinicio controlado del estado del evento/ventas cuando se reconfigura.
- Gestión post-venta (extra): cancelación “soft delete” (status=CANCELLED) y modificación de ventas (admin), dejando rastro en auditoría.

3.3 Validaciones y control de errores (calidad)

Se aplican validaciones para asegurar integridad:

- rangos: aforo 1–500 y precio 1–10000 CHF (enunciado + validación en código),
- códigos de descuento restringidos a la lista definida,
- control de asiento ocupado (no permitir duplicados),
- cierre automático cuando vendidos $\geq$ aforo.

3.4 Casos límite contemplados

- Si no hay evento configurado, el sistema fuerza el flujo de configuración antes de vender.
- Cuando no quedan plazas, se bloquean compras y se marca cierre de ventas.
- En cálculos (KPIs/métricas), se separan ventas activas de canceladas para no falsear ingresos ni ocupación.

4. Diseño de la solución (arquitectura + decisiones)

4.1 Flujo general por menús (casos de uso)

El diseño sigue el flujo exigido por el enunciado: configuración (admin) → ventas → cierre/informes (admin). Primero, el administrador introduce el código secreto fijo CEAC4013 para poder configurar el evento (nombre, licencia, aforo y precio unitario). Tras configurar, el sistema pasa al menú de ventas con opciones de listar disponibilidad y reservar; y finalmente, al cerrar ventas (por opción o por aforo completo), se pide de nuevo el código secreto y se abre el menú de informes (informe total + ordenaciones asc/desc). Además, el aforo total y plazas disponibles deben estar siempre visibles durante la operación.

4.2 Diseño de datos y estructuras

En C, el diseño se apoya en estructuras simples y eficientes: una configuración del evento (campos básicos) y un registro por venta (cliente, edad, asiento/posición, descuento y total pagado). La ocupación se controla con un array de “asientos ocupados” (disponibilidad instantánea), y las ventas se almacenan en un array hasta completar aforo (máx. 500), lo que cumple el enfoque de arrays y facilita el cálculo de KPIs e informes.

En Python, el diseño mantiene el mismo dominio pero añade persistencia en SQLite, lo que permite trazabilidad (ventas, auditoría, métricas) y consultas más robustas. La estructura se organiza por responsabilidades (DB/servicio/UI), mejorando mantenibilidad y escalabilidad.

4.3 Algoritmos y lógica clave

Para cumplir el requisito de ordenación, se usa bubble sort para mostrar ventas ascendente/descendente por precio (suficiente para el tamaño máximo de datos). Los KPIs se calculan agregando ventas (tickets e ingresos) y separando el estado cuando existen cancelaciones (en Python).

4.4 Validación y control de errores

El diseño contempla validaciones desde entrada: rangos de aforo y precio, edad mínima, asiento libre, y códigos de descuento válidos. En Python se formaliza con reglas constantes (p. ej., `DISCOUNT_CODES`, límites y confirmación explícita de compra para registrar o no la venta).

4.5 Valor agregado (diseño de extras)

Siguiendo “trabajo adicional”, se incluyen funcionalidades acordes al negocio: confirmación de compra, cancelación (soft delete), modificación con recálculo, auditoría, métricas y exportación CSV/JSON para reporte y control operativo del evento.

5. Implementación en C

5.1 Estructura del programa y modularización

La versión en C se organiza de forma procedimental y modular, separando claramente: (1) utilidades de entrada, (2) lógica de negocio (zonas, descuentos y pricing), (3) reporting/ordenación y (4) flujos principales (configuración, ventas y cierre). Esta división facilita la lectura y el mantenimiento, y demuestra “programación estructurada” y “modularización” en un entorno CLI.

5.2 Estructuras de datos, arrays y control de aforo

El dominio se modela con struct EventConfig (nombre del evento, licencia, aforo, precio base) y struct Sale (nombre, edad, índice de asiento, zona, descuento y total pagado). La disponibilidad se controla con un array de ocupación (taken[500]) que marca 0/1 por posición, lo que permite validar rápidamente si un asiento está libre. Las ventas se almacenan en un array dinámico sales reservado con calloc(capacity, sizeof(Sale)), ajustando memoria al aforo real.

5.3 Control de flujo y validación de entradas

El programa implementa menús con bucles while y opciones numéricas, apoyándose en funciones robustas como read_int/read_double para validar rangos (aforo 1–500, precio 1–10000, edad, etc.). La autenticación admin se aplica con require_admin_code() usando el código fijo CEAC4013. La reserva (reserve_flow) incluye confirmación explícita (ask_yes_no) antes de registrar una venta.

5.4 Informes, KPIs y ordenación

Al cerrar ventas (por opción o aforo completo), se calculan KPIs (ocupación, ingresos, descuentos, edad media, ventas por zona). Para cumplir el requisito de “asc/desc”, se crea una copia del array de ventas y se ordena con bubble sort por paid, mostrando listados sin alterar el histórico original.

6. Implementación en Python

6.1 Estructura y arquitectura

La versión en Python se implementa como una aplicación CLI con dos modos: (1) modo menú interactivo sin argumentos y (2) modo subcomandos (configure, reserve, report, metrics, export, audit, test). Esta decisión permite demostrar usabilidad para un usuario “de negocio” (menús) y, a la vez, facilitar pruebas y automatización (subcomandos).

6.2 Persistencia con SQLite y modelo de datos

A diferencia del enfoque en memoria de C, aquí las ventas y la configuración del evento se guardan en SQLite, lo que mejora la trazabilidad y permite consultas fiables (ventas activas, asientos ocupados, histórico). Por ejemplo, se controla que un asiento no pueda estar duplicado consultando ventas ACTIVE por seat_index, y se separan ventas activas de canceladas.

6.3 Reglas de negocio, validaciones y seguridad de acceso

La lógica del dominio se concentra en un servicio (SunriseService) que valida rangos (aforo 1-500, precio 1-10000), normaliza descuentos y aplica el requisito de código admin fijo mediante `require_admin`.

La compra incluye resumen + confirmación; si no se confirma, no se registra la venta.

6.4 Funcionalidades extra (valor agregado)

Se añaden funciones coherentes con el negocio: cancelación soft delete (status=CANCELLED), modificación de ventas con recálculo y control de asientos, auditoría, métricas y exportación CSV/JSON desde el menú de informes.

Finalmente, se incluyen tests unitarios para validar casos críticos (cancelar, modificar y asiento único).

7. Comparación C vs Python

La solución en C y en Python resuelve el mismo problema de negocio (configurar un evento, vender/reservar con aforo limitado, aplicar descuentos y generar informes), pero lo hace con enfoques técnicos distintos que afectan a la calidad, mantenibilidad y evolución del sistema.

En C, el estilo es principalmente procedimental y orientado al control directo: se trabaja con `struct` y `arrays` para representar configuración, disponibilidad y ventas. Esto da mucha eficiencia y previsibilidad, pero obliga a gestionar manualmente aspectos críticos como la memoria (`malloc/calloc/free`), el manejo de cadenas y la validación de entrada para evitar errores (desbordes, lecturas inválidas, estados inconsistentes). El control de flujo mediante menús y bucles es claro y rápido para un terminal de taquilla, y la ordenación (bubble sort) es suficiente para un aforo pequeño (≤ 500), aunque no sería óptima a gran escala.

En Python, el enfoque es más alto nivel y escalable: se utilizan estructuras dinámicas y, sobre todo, persistencia en SQLite, lo que mejora la trazabilidad (histórico, auditoría, exportaciones) y permite reanudar el estado del sistema sin pérdida. El manejo de errores con excepciones (por ejemplo, reglas de dominio vs errores de permisos) reduce complejidad y hace el código más legible. Además, el sistema puede incorporar extras “profesionales” con menos coste: métricas, export CSV/JSON, cancelación soft delete, y tests unitarios, lo que aumenta la confianza en cambios futuros.

En conclusión, C es ideal para un flujo simple, rápido y controlado en terminal; Python facilita crecimiento, trazabilidad y calidad mediante persistencia, testing y extensibilidad.

8. Pruebas y verificación

La verificación se planteó con una estrategia mixta: tests automáticos en Python y pruebas funcionales/manuales en C, asegurando que cada requisito del sistema tenga al menos un caso de prueba asociado (matriz *requisito* → *prueba* → *resultado esperado*).

En Python, se incluyen tests unitarios con unittest para validar los casos críticos del dominio y evitar regresiones:

- Autenticación/acciones permitidas (solo admin puede acceder a ciertos menús/acciones),
- Creación y registro correcto de ventas,
- Reglas de negocio (descuentos válidos, límites de entradas, asientos únicos),
- Operaciones extra (cancelación “soft delete” y modificación con recálculo),
- Coherencia de reportes/métricas tras cambios.

En C, la verificación se realiza mediante pruebas funcionales ejecutando el programa y comprobando salidas de consola y estados internos:

- Reserva de asiento libre vs intento de asiento ocupado,
- Descuento inválido (no aplica),
- Confirmación “no” (no registra venta),
- Cierre automático al alcanzar el aforo,
- Informes y ordenación asc/desc correctas.

Esta combinación garantiza correctitud, robustez ante entradas inválidas y evidencia clara para evaluación.

9. Guía de usuario

9.1 Ejecución y uso en C

Compilar el programa con GCC y ejecutar en consola. Al iniciar, introducir el código administrador CEAC4013 para configurar el evento (nombre, licencia, aforo y precio). En el menú de ventas, usar “Ver disponibilidad” y “Reservar ticket” (con confirmación). Para finalizar, seleccionar “Cerrar ventas y generar informe” y acceder al menú de informes (asc/desc).

9.2 Ejecución y uso en Python

Inicializar la base de datos y crear usuarios. Después, ejecutar el flujo desde CLI o menú: configurar el evento, crear/regar registrar ventas, consultar disponibilidad y cerrar. Para gestión avanzada: generar reporte/KPIs, exportar a CSV/JSON, revisar auditoría y métricas, y ejecutar test para validar reglas (cancelación/modificación, asientos únicos y cálculos).

10. Apéndice

Apéndice A — Checklist de Requisitos (Trazabilidad)

Requisito	Evidencia en C (dónde)	Evidencia en Python (dónde)	Cómo se prueba	Resultado esperado
Código administrador fijo e inmodificable	#define ADMIN_CODE "CEAC4013" + require_admin_code()	require_admin(...) o función equivalente con "CEAC4013"	Intentar entrar a configuración/informes con código incorrecto	Acceso denegado sin permitir continuar
Configurar evento: nombre	configure_event() pide event_name	configure / configure_event (persistido)	Configurar con texto válido	Evento queda guardado y se muestra en cabecera
Configurar evento: licencia	configure_event() pide license	Campo license en configuración	Configurar con texto válido	Se almacena y aparece en pantalla/informes
Configurar evento: aforo 1..500	read_int("Aforo...",1,500, &cap)	Validación capacity 1..500	Probar aforo 0, 501, y valor válido	Rechaza fuera de rango; acepta válido
Configurar evento: precio 1..10000 CHF	read_double(...,1,10000, &price)	Validación base_price 1..10000	Probar precio 0, 10001, y válido	Rechaza fuera de rango; acepta válido
Mostrar aforo total y plazas disponibles en ejecución	print_event_header(&cfg, available)	Cabecera / status persistente (config + disponibles)	Tras cada acción (listar/reservar)	Se ve total y disponibles siempre

Menú de ventas accesible tras configurar	while(sales_open) + menú ventas	Menú CLI / subcomandos de ventas	Ejecutar tras configurar	Se entra al menú de ventas sin errores
Opción: ver disponibilidad (asientos libres)	list_availability(&cfg, taken)	Consulta de asientos libres (por DB/estado)	Ejecutar opción disponibilidad	Lista correcta de posiciones libres
Opción: reservar ticket	reserve_flow()	reserve / create_sale (o equivalente)	Reservar con datos válidos	Venta registrada correctamente
Reserva pide nombre cliente	read_line("Nombre...")	Input/campo customer_name	Reservar con nombre vacío	No permite registrar si vacío
Reserva pide edad + validación	read_int("Edad...",0,120,&age) + MIN_AGE=14	Validación de edad (si existe en Python)	Probar edad 10 (si MIN_AGE=14)	Operación cancelada o rechazada
Descuentos permitidos: DESC5/DESC10/DESC15	discount_factor()	DISCOUNT_CODES o validación equivalente	Probar "DESC10" y "XYZ"	Aplica descuento válido; inválido = sin descuento
Registrar total pagado por venta	sales[idx].paid = after_disc;	Guardado en DB: paid/total	Crear venta y revisar reporte/export	Total pagado queda almacenado
Elegir posición/asiento o y validar libre	read_int(...seat...) + if(taken[seat])	Validación asiento único (consulta DB/estado)	Reservar 2 veces mismo asiento	Rechaza duplicado (en C obliga a elegir otro)
Confirmación de compra antes de registrar	ask_yes_no("Confirmar compra?")	Confirmación explícita (si implementada)	Responder "N/NO"	No se registra la venta
Cierre automático	if (*sold_count >= cfg->capacity) ...	Regla de cierre por capacidad	Configurar aforo pequeño	Ventas se cierran

cuando se alcanza aforo		(DB/estado)	y llenar	automáticamente
Opción “Cerrar ventas y generar informe”	Menú ventas opción 3	Comando/ menú de cierre + reporte	Elegir cerrar antes de llenar	Se pasa a cierre e informes
Informes requieren admin code	require_admin_code() en cierre	require_admin(...) en report/audit/metrics	Intentar informe con código mal	Bloquea acceso a informes
Informe: total tickets + total dinero	show_report_summary()	report() / KPIs desde DB	Generar informe tras ventas	Totales correctos y coherentes
Listado ventas orden ascendente	bubble_sort_sales(...ascending=1) + print_sales_list()	Ordenación ORDER BY paid ASC o equivalente	Crear varias ventas con precios distintos	Sale ordenado barato→caro
Listado ventas orden descendente	bubble_sort_sales(...ascending=0)	ORDER BY paid DESC o equivalente	Igual que A20	Sale ordenado caro→barato
KPIs ampliados (ocupación, zona, descuentos, edad media)	compute_kpis() + show_report_summary()	KPIs extra si existen (por DB)	Comparar KPIs con ventas realizadas	KPIs consistentes (sumas cuadran)
Persistencia (reiniciar app sin perder datos)	(No aplica en C base)	SQLite: ventas/config persistentes	Crear ventas, cerrar, reabrir	Datos siguen disponibles
Cancelación “soft delete”	(Opcional si existe)	status=CANCEL LED + auditoría	Cancelar venta y generar informe	No cuenta como activa; queda trazada
Export CSV/JSON	(Opcional)	export_csv/ export_json	Exportar y abrir archivo	Archivo válido y consistente con DB
Auditoría	(Opcional)	audit_log +	Ejecutar acción	Se registra

(quién hizo qué)		audit(...)	y revisar auditoría	actor/acción /fecha
Métricas (contadores / tiempos)	(Opcional)	metrics(...)	Ejecutar varias acciones y ver métricas	Contadores aumentan correctamente
Tests unitarios	(Opcional)	unittest en Sunrise2.py	Ejecutar comando test	Tests pasan (OK)

La tabla A demuestra trazabilidad completa: cada requisito tiene implementación en C/Python y un método de prueba verificable.

Apéndice B – Capturas de ejecución / Evidencia de funcionamiento

Python – Ski_ticketing_final_V3

B-PY1 Inicio Base de Datos

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python .\Sunrise.py init-db
✓ DB inicializada en: data\sunrise.sqlite3
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> |
```

B-PY2 Configuración del evento(admin) (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py configure --admin-code CEAC4013 --name "Sunrise Ski Session - Davos Parsenn" --license "DK-2026-01" --capacity 10 --base 100
✓ Evento configurado.
```

B-PY3 Ver disponibilidad (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py availability

--- Disponibilidad por zonas (indices libres) ---
Zona A - First Lift Line (+20%): 0 1
Zona B - Acceso estandar (base): 2 3 4 5 6
Zona C - Entrada tardia (-10%): 7 8 9
-----
```

B-PY4 — Reserva 2 tickets con descuento + confirmación (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py reserve --qty 1 --name "Jose Palacios" --age 24 --discount "DESC5" --seats 0 --confirm

Resumen ticket: asiento=0 | zona=A - First Lift Line (+20%) | precio=114.00 CHF
Descuento: DESC5 (5%)
✓ Compra confirmada. Sales IDs: [1]
```

B-PY5 Reserva 2 tickets con descuento (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py reserve --qty 2 --name "Cliente 2" --age 30 --discount "DESC10" --seats 3,8 --confirm

Resumen ticket: asiento=3 | zona=B - Acceso estandar (base) | precio=90.00 CHF
Descuento: DESC10 (10%)

Resumen ticket: asiento=8 | zona=C - Entrada tardia (-10%) | precio=81.00 CHF
Descuento: DESC10 (10%)
✅ Compra confirmada. Sales IDs: [2, 3]
```

B-PY6 — Listar ventas (sin canceladas) (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py list-sales
```

ID	Nombre	Edad	Zona	Desc	Pagado	Estado
1	Jose Palacios	24	A	DESC5	114.00	ACTIVE
2	Cliente 2	30	B	DESC10	90.00	ACTIVE
3	Cliente 2	30	C	DESC10	81.00	ACTIVE

B-PY7 — Cancelar una venta (SOFT DELETE) (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py cancel-sale --admin-code CEAC4013 --id 2 --reason "Cliente se equivoco de dia"
✅ Venta cancelada (status=CANCELLED).
```

B-PY8 — Modificar una venta existente (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py list-sales --sort desc --include-cancelled
```

ID	Nombre	Edad	Zona	Desc	Pagado	Estado
1	Jose Palacios	24	A	DESC5	114.00	ACTIVE
2	Cliente 2	30	B	DESC10	90.00	CANCELLED
3	Cliente 2	30	C	DESC10	81.00	ACTIVE

B-PY9 — Listar ventas ORDEN ASC (bubble sort) (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py list-sales --sort asc --include-cancelled
```

ID	Nombre	Edad	Zona	Desc	Pagado	Estado
3	Cliente 2	30	C	DESC10	81.00	ACTIVE
2	Cliente 2	30	B	DESC10	90.00	CANCELLED
1	Jose Palacios	24	A	DESC5	114.00	ACTIVE

B-PY10 — Modificar una venta existente (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py update-sale --admin-code CEAC4013 --id 1 --name "Jose P." --age 25 --seat 6 --discount DESC5
✅ Venta modificada.
```

B-PY11 — Cerrar ventas (admin) (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py close-sales --admin-code CEAC4013
✅ Ventas cerradas.
```

B-PY12 — Informe / KPIs (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py report
{
  "tickets_activos": 2,
  "tickets_cancelados": 1,
  "ingresos_totales_activos": 176.0,
  "ocupacion_pct": 20.0,
  "ventas_por_zona": {
    "A": 0,
    "B": 1,
    "C": 1
  },
  "ingresos_por_zona": {
    "A": 0.0,
    "B": 95.0,
    "C": 81.0
  },
  "uso_descuentos": {
    "DESC5": 1,
    "DESC10": 1,
    "DESC15": 0
  },
  "edad_media": 27.5
}
```

B-PY13 — Métricas (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py metrics
{
  "generated_at": "2026-01-08T19:26:21+00:00",
  "gauges": {
    "sold_active": 2,
    "available": 8,
    "sales_open": false
  },
  "counters": {
    "attempted_sales": 3,
    "availability_views": 1,
    "confirmed_sales": 3,
    "discount_desc10_used": 2,
    "discount_desc5_used": 1,
    "event_configured": 1,
    "sale_cancelled": 1,
    "sale_modified": 1,
    "sales_closed": 1,
    "sold_zone_a": 1,
    "sold_zone_b": 1,
    "sold_zone_c": 1
  },
  "paid_total_stats": {
    "count": 3,
    "p50": 90.0,
    "p95": 111.6,
    "min": 81.0,
    "max": 114.0
  }
}
```

B-PY14 — Exportar CSV y JSON (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python .\Sunrise.py export --csv .\exports\sales.csv --json .\exports\sales.json
✓ Exportado CSV: exports\sales.csv
✓ Exportado JSON: exports\sales.json
```

B-PY15 — Ver auditoría (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py audit
- [2026-01-08T19:35:51+00:00] export | {"csv": "exports\\sales.csv", "json": "exports\\sales.json", "rows": 3}
- [2026-01-08T19:16:48+00:00] sales_closed | {}
- [2026-01-08T19:15:41+00:00] sale_modified | {"sale_id": 1, "before": {"id": 1, "customer_name": "Jose Palacios", "age": 24, "seat_index": 0, "discount_code": "DESC5", "paid_total": 114.0, "created_at": "2026-01-08T18:52:55+00:00", "updated_at": "2026-01-08T18:52:55+00:00", "status": "ACTIVE", "cancelled_at": null}, "after": {"id": 1, "customer_name": "Jose P.", "age": 25, "seat_index": 6, "discount_code": "DESC5", "paid_total": 95.0, "created_at": "2026-01-08T18:52:55+00:00", "updated_at": "2026-01-08T19:15:41+00:00", "status": "ACTIVE", "cancelled_at": null}}
- [2026-01-08T19:06:22+00:00] sale_cancelled | {"sale_id": 2, "reason": "Cliente se equivoco de dia"}
- [2026-01-08T18:54:17+00:00] purchase_confirmed | {"sale_id": 3, "seat": 8, "zone": "C", "paid": 81.0}
- [2026-01-08T18:54:17+00:00] purchase_confirmed | {"sale_id": 2, "seat": 3, "zone": "B", "paid": 90.0}
- [2026-01-08T18:54:17+00:00] purchase_attempt | {"qty": 2, "name": "Cliente 2", "age": 30, "discount": "DESC10", "seats": [3, 8]}
- [2026-01-08T18:52:55+00:00] purchase_confirmed | {"sale_id": 1, "seat": 0, "zone": "A", "paid": 114.0}
- [2026-01-08T18:52:55+00:00] purchase_attempt | {"qty": 1, "name": "Jose Palacios", "age": 24, "discount": "DESC5", "seats": [0]}
- [2026-01-08T18:50:38+00:00] availability | {"free": 10}
- [2026-01-08T18:49:01+00:00] configure_event | {"event_name": "Sunrise Ski Session - Davos Parsenn", "license": "DK-2026-01", "capacity": 10, "base_price": 100.0, "configured": 1, "sales_open": 1}
```


B-PY16 — Tests (CLI)

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> python Sunrise.py test
test_insert_and_cancel (__main__.run_tests.<locals>.SunriseTests.test_insert_and_cancel) ...
Resumen ticket: asiento=0 | zona=A - First Lift Line (+20%) | precio=108.00 CHF
    Descuento: DESC10 (10%)
ok
test_modify_seat_recalc_price (__main__.run_tests.<locals>.SunriseTests.test_modify_seat_recalc_price) ...
Resumen ticket: asiento=0 | zona=A - First Lift Line (+20%) | precio=120.00 CHF
    Descuento: -
ok
test_unique_active_seat (__main__.run_tests.<locals>.SunriseTests.test_unique_active_seat) ...
Resumen ticket: asiento=1 | zona=A - First Lift Line (+20%) | precio=120.00 CHF
    Descuento: -
ok
-----
Ran 3 tests in 0.006s
OK
```

C (ski_ticketing_final_v3.c)

B-C1 — Ejecución + acceso admin

```
PS F:\UNIVERSIDAD\BAC\Level 4\Software Development\Trabajo> .\output\ski_ticketing_final_v3.exe --actor admin

=====
  Sunrise Ski Session - Davos
=====
Evento: (no configurado)
Licencia: (no configurada)
Aforo: 1 | Precio unitario base: 1 CHF
Vendidas: 0 | Disponibles: 1
Actualizado: 1970-01-01 00:00:00 UTC
=====

=== Menú Principal ===
Rol: ADMIN
1) Configurar evento (admin)
2) Ventas
3) Informes (admin)
0) Salir
Opción: |
```

B-C2 — Configuración del evento (entradas)

```
Introduce ADMIN code: CEAC4013
✅ Acceso concedido.
Nombre del evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo (1..500): 10
Precio unitario CHF (1..10000): 100
✅ Evento configurado.

=====
  Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 0 | Disponibles: 10
Actualizado: 2026-01-09 04:55:22 UTC
=====
```

B-C3 — Menú de ventas + disponibilidad

```

--- Menú Ventas ---
1) Listar disponibilidad
2) Reservar ticket
3) Cancelar venta (extra)
9) Cerrar y generar informe (admin)
0) Volver
Opción: 1

★ Disponibilidad (asientos libres):
 1  2  3  4  5  6  7  8  9 10
Total libres: 10

=====
Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 0 | Disponibles: 10
Actualizado: 2026-01-09 04:55:22 UTC
=====

```

B-C4 — Reserva de ticket (1 ticket, sin descuento)

```

Opción: 2

★ Disponibilidad (asientos libres):
 1  2  3  4  5  6  7  8  9 10
Total libres: 10
Nombre: José Palacios
Edad: 24
Código descuento (DESC5/DESC10/DESC15 o ENTER):
Elige posición/asiento: 1

📌 (Extra) ¿Quieres elegir tipo de ticket según cuándo subes?
1) Sí (SUNRISE/MORNING/AFTERNOON/FULLDAY)
0) No (usar precio unitario base del evento)
Elige 1/0: 1

Tipos activos:
1) SUNRISE - Sunrise (primera hora) (80 CHF)
2) MORNING - Morning (mañana) (110 CHF)
3) AFTERNOON - Afternoon (tarde) (90 CHF)
4) FULLDAY - Full Day (día completo) (140 CHF)
ID tipo ticket: 1
Fecha de esquí (YYYY-MM-DD o ENTER): 2026/01/06

📄 Resumen
Nombre: José Palacios | Edad: 24
Asiento: 1
Precio unitario: 80 CHF
Subtotal: 80 CHF
Descuento: ninguno
TOTAL A PAGAR: 80 CHF
Confirmar compra (S/N): S
✅ Venta confirmada. Vendidas=1 | Disponibles=9

=====
Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC

```

B-C5 — Reserva con descuento

```
Opción: 2
★ Disponibilidad (asientos libres):
  2  3  4  5  6  7  8  9 10
Total libres: 9
Nombre: Carlota Ciudad
Edad: 30
Código descuento (DESC5/DESC10/DESC15 o ENTER): DESC10
Elige posición/asiento: 2

🟡 (Extra) ¿Quieres elegir tipo de ticket según cuándo subes?
1) Sí (SUNRISE/MORNING/AFTERNOON/FULLDAY)
0) No (usar precio unitario base del evento)
Elige 1/0: 1

Tipos activos:
1) SUNRISE – Sunrise (primera hora) (80 CHF)
2) MORNING – Morning (mañana) (110 CHF)
3) AFTERNOON – Afternoon (tarde) (90 CHF)
4) FULLDAY – Full Day (día completo) (140 CHF)
ID tipo ticket: 2
Fecha de esquí (YYYY-MM-DD o ENTER): 2026/01/09

📄 Resumen
Nombre: Carlota Ciudad | Edad: 30
Asiento: 2
Precio unitario: 110 CHF
Subtotal: 110 CHF
Descuento DESC10 (10%): -11 CHF
TOTAL A PAGAR: 99 CHF
Confirmar compra (S/N): S
✅ Venta confirmada. Ventidas=2 | Disponibles=8

=====
    Sunrise Ski Session – Davos
=====
Evento: Sunrise Ski Session – Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Ventidas: 2 | Disponibles: 8
Actualizado: 2026-01-09 04:55:22 UTC
```

B-C6 — Cancelar Ventas

```
--- Menú Ventas ---
1) Listar disponibilidad
2) Reservar ticket
3) Cancelar venta (extra)
9) Cerrar y generar informe (admin)
0) Volver
Opción: 3
ID venta a cancelar: 2
Confirmar cancelación (S/N): s
✅ Venta cancelada (asiento liberado).

=====
    Sunrise Ski Session – Davos
=====
Evento: Sunrise Ski Session – Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Ventidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====
```

B-C7 — Cerrar ventas y generar informe

```

--- Menú Ventas ---
1) Listar disponibilidad
2) Reservar ticket
3) Cancelar venta (extra)
9) Cerrar y generar informe (admin)
0) Volver
Opción: 9
Introduce ADMIN code: CEAC4013
✓ Acceso concedido.

=====
  Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====

--- Menú Informes (Admin) ---
1) Ver informe (tickets + dinero)
2) Ver ventas ascendente (barato→caro)
3) Ver ventas descendente (caro→barato)
4) Export (CSV/JSON + audit.csv)
5) Métricas (p50/p95)
0) Volver
Opción:

```

B-C8 — Informe (KPIs)

```

--- Menú Informes (Admin) ---
1) Ver informe (tickets + dinero)
2) Ver ventas ascendente (barato→caro)
3) Ver ventas descendente (caro→barato)
4) Export (CSV/JSON + audit.csv)
5) Métricas (p50/p95)
0) Volver
Opción: 1

📄 INFORME
Tickets vendidos: 1
Total dinero: 80 CHF

=====
  Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====

```

B-C9 — Ventas ordenadas ascendente

```

--- Menú Informes (Admin) ---
1) Ver informe (tickets + dinero)
2) Ver ventas ascendente (barato→caro)
3) Ver ventas descendente (caro→barato)
4) Export (CSV/JSON + audit.csv)
5) Métricas (p50/p95)
0) Volver
Opción: 2

📄 Ventas ASC (barato→caro)

ID | Seat | Buyer                | Age | Price | Final | Disc | Status | Cr
eated (UTC)
-----+-----+-----+-----+-----+-----+-----+-----+-----
1  | 1    | José Palacios        | 24  | 80    | 80    | 0    | CONFIR | 2026
-01-09 05:22:32
2  | 2    | Carlota Ciudad       | 30  | 110   | 99    | 11   | CANCEL | 202
6-01-09 05:26:02

=====
Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====

```

B-C10 — Ventas ordenadas descendente

```

--- Menú Informes (Admin) ---
1) Ver informe (tickets + dinero)
2) Ver ventas ascendente (barato→caro)
3) Ver ventas descendente (caro→barato)
4) Export (CSV/JSON + audit.csv)
5) Métricas (p50/p95)
0) Volver
Opción: 3

📄 Ventas DESC (caro→barato)

ID | Seat | Buyer                | Age | Price | Final | Disc | Status | Cr
eated (UTC)
-----+-----+-----+-----+-----+-----+-----+-----+-----
2  | 2    | Carlota Ciudad       | 30  | 110   | 99    | 11   | CANCEL | 202
6-01-09 05:26:02
1  | 1    | José Palacios        | 24  | 80    | 80    | 0    | CONFIR | 2026
-01-09 05:22:32

=====
Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====

```

B-C11 Export (CSV/JSON + audit.csv)

```

--- Menú Informes (Admin) ---
1) Ver informe (tickets + dinero)
2) Ver ventas ascendente (barato→caro)
3) Ver ventas descendente (caro→barato)
4) Export (CSV/JSON + audit.csv)
5) Métricas (p50/p95)
0) Volver
Opción: 4
✅ Export completado:
- exports/sales.csv
- exports/sales.json
- exports/audit.csv

=====
Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====

```

B-C12 Metricas

```

--- Menú Informes (Admin) ---
1) Ver informe (tickets + dinero)
2) Ver ventas ascendente (barato→caro)
3) Ver ventas descendente (caro→barato)
4) Export (CSV/JSON + audit.csv)
5) Métricas (p50/p95)
0) Volver
Opción: 5

📊 Métricas
Counters:
- admin_config_event: 1
- sales_list_availability: 1
- sales_reserve: 2
- sales_cancel_sale: 1
- reports_ver_informe: 1
- reports_asc: 1
- reports_desc: 1
- reports_export: 1
- reports_metrics: 1

Timings (segundos):
Main:    n=1 | p50=239.2790 | p95=239.2790
Sales:   n=0 | p50=0.0000 | p95=0.0000
Reports: n=0 | p50=0.0000 | p95=0.0000

=====
Sunrise Ski Session - Davos
=====
Evento: Sunrise Ski Session - Davos Parsenn
Licencia: DK-2026-01
Aforo: 10 | Precio unitario base: 100 CHF
Vendidas: 1 | Disponibles: 9
Actualizado: 2026-01-09 04:55:22 UTC
=====

```

APÉNDICE C — Diagrama de flujo / Pseudocódigo

C.1 Flujo general (C — Sunrise Ski Session)

Diagrama de flujo

Inicio

1. Mostrar banner del sistema
2. Configurar evento (ADMIN)
 - Pedir ADMIN_CODE
 - Capturar: nombre, licencia, aforo (1..500), precio (1..10000)
 - Inicializar arrays / estructuras (asientos libres, ventas)
3. Entrar en Bucle principal (Menú de ventas) mientras ventas estén abiertas:
 - Mostrar cabecera de evento (disponibles, precio base, etc.)
 - Mostrar opciones:
 - (1) Ver disponibilidad
 - (2) Reservar ticket(s)
 - (3) Cerrar ventas y generar informe
 - (0) Salir
 - Ejecutar opción
 - Si aforo completo → cerrar ventas automáticamente
4. Cierre:
 - Pedir ADMIN_CODE para acceder a informes
 - Mostrar resumen KPIs + submenú de informes (ordenación, listados)
5. Liberar memoria y terminar

C.2 Flujo general (Python — Ticketing System CLI)

Inicio

1. Parsear argumentos CLI (argparse)

2. Conectar/crear DB SQLite (init-db o auto-init)
3. Si comando no requiere login (init-db, test, add-user, list-users) → ejecutar y salir
4. Si comando requiere login:
 - Autenticar usuario (username + password)
 - Aplicar RBAC según rol:
 - requester: solo sus tickets, cerrar/reabrir, comentar/etiquetar sus tickets
 - agent/admin: asignar, transicionar, reportes, export, métricas
5. Ejecutar caso de uso: create-ticket / assign / transition / comment / tag / report / metrics / export
6. Registrar auditoría + métricas (counters y timings)
7. Salida por terminal y finalizar

Apéndice D — Esquema de Base de Datos (SQLite)

Este apéndice documenta **la estructura SQLite** usada por la aplicación (C y, como variante, Python), explicando **tablas, campos, restricciones, índices, “migraciones”** y cómo todo ello soporta los requisitos (ventas, control de aforo, cancelaciones, auditoría, KPIs y exportación).

D.1 Objetivo del esquema

La base de datos se usa para:

- **Persistir la configuración del evento** (nombre, licencia, aforo, precio).
- Registrar **ventas confirmadas** y permitir **cancelación (soft delete)** sin perder histórico.
- Mantener un catálogo de **tipos de ticket y descuentos** (extras).
- Registrar un **log de auditoría** para evidencias y trazabilidad.
- Permitir **reporting/KPIs** (ocupación, ingresos, ventas por tipo/estado, etc.).
- Mejorar rendimiento con **índices** (búsqueda por estado/fecha) y una **restricción única parcial** para no vender dos veces el mismo asiento.

D.2 Configuración SQLite (PRAGMAs)

Al abrir la DB, se aplican ajustes (C):

- PRAGMA foreign_keys = ON;
- PRAGMA journal_mode = WAL; (mejor concurrencia/escrituras)
- PRAGMA synchronous = NORMAL; (equilibrio rendimiento/seguridad)
- PRAGMA temp_store = MEMORY;
- PRAGMA busy_timeout = 5000;

Esto ayuda a que el sistema sea más robusto ante accesos repetidos y reduzca “locks”.

D.3 Esquema principal (Aplicación en C)

A continuación se describe el esquema real del archivo `ski_ticketing_final_v3.c` (función `db_schema_migrate()`).

D.3.1 Tabla meta (versionado / migraciones)

Propósito: guardar metadatos como la versión del esquema para futuras migraciones.

- k TEXT PRIMARY KEY
- v TEXT NOT NULL

Ejemplo: ('schema_version','3').

D.3.2 Tabla event (configuración del evento)

Propósito: almacenar la configuración del evento (solo **1 fila**, id=1).

Campos:

- id INTEGER PRIMARY KEY CHECK(id=1) → fuerza una única fila
- name TEXT NOT NULL
- license TEXT NOT NULL
- capacity INTEGER NOT NULL CHECK(capacity BETWEEN 1 AND 500)
- unit_price INTEGER NOT NULL CHECK(unit_price BETWEEN 1 AND 10000)
- created_at TEXT NOT NULL
- updated_at TEXT NOT NULL

Reglas clave (integridad):

- **Aforo** limitado 1..500.

- **Precio** limitado 1..10000.
 - Siempre existe una fila inicial “vacía” (INSERT OR IGNORE), evitando nulos.
-

D.3.3 Tabla `ticket_types` (extra)

Propósito: catálogo de tipos de ticket (p.ej. sunrise/morning/full day).

Campos:

- `id` INTEGER PRIMARY KEY AUTOINCREMENT
- `code` TEXT UNIQUE NOT NULL (identificador corto, p.ej. FULLDAY)
- `label` TEXT NOT NULL (nombre humano)
- `price_chf` INTEGER NOT NULL CHECK(`price_chf` BETWEEN 1 AND 10000)
- `active` INTEGER NOT NULL DEFAULT 1 CHECK(`active` IN (0,1))

Ventaja: permite ampliar el sistema sin tocar la lógica base (añadir tipos desde DB).

D.3.4 Tabla `discounts` (extra)

Propósito: catálogo de descuentos configurables.

Campos:

- `code` TEXT PRIMARY KEY
- `kind` TEXT NOT NULL CHECK(`kind` IN ('PERCENT','FIXED'))
- `value` INTEGER NOT NULL CHECK(`value` BETWEEN 1 AND 10000)
- `min_qty` INTEGER NOT NULL DEFAULT 1 CHECK(`min_qty` BETWEEN 1 AND 4)
- `expires_at` TEXT NOT NULL DEFAULT ''
- `active` INTEGER NOT NULL DEFAULT 1 CHECK(`active` IN (0,1))

Reglas:

- Descuento porcentual o fijo.
 - Control mínimo de tickets por compra (1..4) para aplicar descuentos por volumen.
-

D.3.5 Tabla `sales` (ventas + estado + cancelación)

Propósito: registrar cada venta con detalle suficiente para auditoría y KPIs.

Campos principales:

- id INTEGER PRIMARY KEY AUTOINCREMENT
- seat_no INTEGER NOT NULL CHECK(seat_no BETWEEN 1 AND 500)
- buyer TEXT NOT NULL
- buyer_age INTEGER NOT NULL CHECK(buyer_age BETWEEN 0 AND 130)
- qty INTEGER NOT NULL CHECK(qty BETWEEN 1 AND 4)
- ticket_type_id INTEGER (*referencia lógica a ticket_types.id*)
- visit_date TEXT NOT NULL DEFAULT ''
- unit_price INTEGER NOT NULL CHECK(unit_price BETWEEN 1 AND 10000)
- base_total INTEGER NOT NULL
- discount_code TEXT NOT NULL DEFAULT ''
- discount_percent INTEGER NOT NULL DEFAULT 0
- discount_kind TEXT NOT NULL DEFAULT ''
- discount_value INTEGER NOT NULL DEFAULT 0
- discount_amount INTEGER NOT NULL DEFAULT 0
- final_total INTEGER NOT NULL
- status TEXT NOT NULL CHECK(status IN ('CONFIRMED','CANCELED'))
- created_at TEXT NOT NULL
- canceled_at TEXT (*solo si se cancela*)

Puntos de matrícula:

- Se implementa **soft delete**: al cancelar no se borra la fila, se marca como CANCELED y se registra canceled_at.
- Se guarda “antes y después” de descuento: base_total, discount_amount, final_total.

D.3.6 Tabla audit (auditoría)

Propósito: registrar acciones relevantes (configurar evento, venta confirmada, cancelación, exportación, etc.).

Campos:

- id INTEGER PRIMARY KEY AUTOINCREMENT

- actor TEXT NOT NULL (quién ejecuta: admin/usuario)
- action TEXT NOT NULL
- detail TEXT NOT NULL (texto/JSON simplificado)
- created_at TEXT NOT NULL

Esto permite **evidencia directa** en la memoria: “qué se hizo, cuándo y por quién”.

D.4 Índices y rendimiento

Índices definidos en el esquema C:

- CREATE INDEX idx_sales_status ON sales(status);
→ listar ventas confirmadas/canceladas rápidamente.
- CREATE INDEX idx_sales_created ON sales(created_at);
→ reportes por fecha y orden cronológico.
- CREATE INDEX idx_audit_created ON audit(created_at);
→ auditoría reciente sin escaneo completo.

Índice único parcial (detalle clave)

- CREATE UNIQUE INDEX ux_sales_seat_confirmed ON sales(seat_no) WHERE status='CONFIRMED';

Qué garantiza:

- Un mismo asiento **no puede** estar confirmado dos veces.
- Si una venta se **cancela**, el asiento queda libre (la fila sigue existiendo, pero ya no cuenta para la unicidad).

Esto es una solución de **nivel alto** porque combina integridad + soporte de cancelaciones sin perder histórico.

D.5 Relaciones e integridad (modelo conceptual)

Aunque en SQL no se declara una FK explícita en sales.ticket_type_id, conceptualmente:

- event → (1) define el evento
- ticket_types → (N) tipos disponibles
- discounts → (N) descuentos disponibles
- sales → (N) ventas vinculadas al evento y opcionalmente a un tipo de ticket y/o descuento

- audit → (N) eventos de auditoría

Relaciones conceptuales:

- sales.ticket_type_id → ticket_types.id (opcional)
- sales.discount_code → discounts.code (opcional, validado por lógica)

D.6 Versionado / migración del esquema

Se usa:

- meta(schema_version) para conocer la versión del esquema.
- CREATE TABLE IF NOT EXISTS ... para que el programa pueda ejecutarse aunque la DB no exista.
- INSERT OR IGNORE para datos semilla (event inicial, ticket_types, discounts) sin duplicados.

Esto demuestra buenas prácticas de *deploy* y persistencia.

